

ASSESSMENT TOOL CO2 AT2

SUTDENT NAME: Shaik Ahamed Ali

REG. NO: 192425106

COURSE CODE: MLA0302

COURSE NAME: REINFORCEMENT LEARNING

1. Design and develop a Reinforcement Learning framework a ride-sharing company uses Reinforcement Learning to match drivers with passengers efficiently. Apply RL concepts to define the state space, action space, and reward function to minimizewaiting time and improve service quality.

Introduction

A ride-sharing company uses Reinforcement Learning (RL) to match drivers with passengers efficiently. The RL agent learns the best driver assignment policy by interacting with the environment and receiving rewards based on the quality of service. The objective is to minimize passenger waiting time, improve driver utilization, and maximize customer satisfaction.

RL Framework

Agent

The ride-matching system that selects the most suitable driver.

Environment

The ride-sharing platform consisting of drivers, passengers, road network, and traffic conditions.

State Space (S)

The state represents the current condition of the system.

- Driver location
- Passenger location
- Driver availability
- Traffic condition
- Passenger waiting time

Action Space (A)

The possible actions of the RL agent are:

- Assign Driver 1
- Assign Driver 2
- Assign Driver 3
- Delay assignment until a better driver becomes available

Reward Function (R)

Situation	Reward
Passenger assigned immediately	+10
Ride completed successfully	+20
Short pickup distance	+5
Long passenger waiting time	-10
Ride cancellation	-20

Learning Process

The RL agent observes the current state, selects a driver based on its policy, receives a reward after the ride outcome, and updates its policy. Over repeated interactions, the agent learns the optimal driver assignment strategy.

CODE :

```
import numpy as np
import random

# -----
# Ride Sharing Q-Learning
# -----

states = ["Zone A", "Zone B", "Zone C", "Zone D"]

actions = ["Driver 1", "Driver 2", "Driver 3"]

num_states = len(states)
num_actions = len(actions)

# Reward Matrix
reward_matrix = np.array([
    [10, 5, -5], # Zone A
    [-5, 10, 5], # Zone B
    [5, -5, 10], # Zone C
    [10, -5, 5]  # Zone D
])

# Initialize Q-table
Q = np.zeros((num_states, num_actions))

# Hyperparameters
alpha = 0.1
gamma = 0.9
epsilon = 0.2

episodes = 500

for episode in range(episodes):

    state = random.randint(0, num_states-1)

    # Epsilon-Greedy Action Selection
    if random.uniform(0,1) < epsilon:
        action = random.randint(0, num_actions-1)
```

```

else:
    action = np.argmax(Q[state])

reward = reward_matrix[state][action]

next_state = random.randint(0, num_states-1)

Q[state][action] = Q[state][action] + alpha * (
    reward + gamma * np.max(Q[next_state]) - Q[state][action]
)

print("\nLearned Q-Table\n")
print(np.round(Q,2))

print("\nOptimal Driver Assignment\n")

for i in range(num_states):
    best_driver = np.argmax(Q[i])
    print(states[i], "->", actions[best_driver])Result

```

OUTPUT :

```

Learned Q-Table

[[59.12 23.94 14.48]
 [22.63 56.72 22.46]
 [52.62  9.41 27.66]
 [57.36 19.97 13.27]]

Optimal Driver Assignment

Zone A -> Driver 1
Zone B -> Driver 2
Zone C -> Driver 1
Zone D -> Driver 1

```

RESULT :

The RL framework minimizes passenger waiting time, improves driver utilization, and enhances overall service quality.

2. An e-learning platform uses Reinforcement Learning to recommend personalized study materials. Recall how exploration and exploitation strategies help balance familiar and new content recommendations.

Introduction

An e-learning platform uses Reinforcement Learning to recommend personalized learning materials according to each student's interests and learning progress. The system balances exploration and exploitation to improve recommendations.

Exploration

Exploration allows the system to recommend new learning materials that the student has not studied before.

- Discovers new topics.
- Prevents repetitive recommendations.
- Collects additional feedback.

Exploitation

Exploitation recommends learning materials that have previously produced good learning outcomes.

- Improves recommendation accuracy.
- Increases student satisfaction.
- Maximizes learning efficiency.

Learning Process

Initially, the system explores different study materials. As more feedback is collected, it gradually exploits the best-performing resources while occasionally exploring new ones.

CODE :

```
import numpy as np
import random

# -----
# E-Learning Recommendation System
# using  $\epsilon$ -Greedy Reinforcement Learning
# -----

states = ["Mathematics",
          "Programming",
          "Machine Learning",
          "Data Science"]

actions = ["Video",
          "PDF Notes",
          "Quiz",
          "Coding Exercise"]

num_states = len(states)
num_actions = len(actions)

# Reward Matrix
reward_matrix = np.array([
    [10, 6, 5, 2],    # Mathematics
    [5, 6, 8, 10],    # Programming
    [6, 5, 10, 9],    # Machine Learning
    [7, 8, 10, 6]     # Data Science
])

Q = np.zeros((num_states, num_actions))

alpha = 0.1
gamma = 0.9
epsilon = 0.2

episodes = 500

for episode in range(episodes):

    state = random.randint(0, num_states - 1)

    #  $\epsilon$ -Greedy Strategy
    if random.uniform(0,1) < epsilon:
        action = random.randint(0, num_actions-1)
        strategy = "Exploration"
    else:
        action = np.argmax(Q[state])
        strategy = "Exploitation"
```

```

reward = reward_matrix[state][action]

next_state = random.randint(0, num_states-1)

Q[state][action] = Q[state][action] + alpha * (
    reward +
    gamma*np.max(Q[next_state]) -
    Q[state][action]
)

print("\nLearned Q-Table\n")
print(np.round(Q,2))

print("\nBest Recommendation for Each Subject\n")

for i in range(num_states):
    best = np.argmax(Q[i])
    print(states[i], "->", actions[best])

```

OUTPUT :

```

Learned Q-Table

[[50.61 15.47 21.75 21.02]
 [21.08 21.66 48.32  5.76]
 [46.69 11.21  6.66 18.31]
 [47.58 21.58 25.9  22.47]]

Best Recommendation for Each Subject

Mathematics -> Video
Programming -> Quiz
Machine Learning -> Video
Data Science -> Video

```

Result

Balancing exploration and exploitation provides personalized recommendations, improves learning outcomes, and increases student engagement.

3. Design and develop a Reinforcement Learning framework a robotic vacuum cleaner uses Reinforcement Learning to clean efficiently while avoiding obstacles. Apply the concept of policy and explain how the value function improves performance over time.

Introduction

A robotic vacuum cleaner uses Reinforcement Learning to clean rooms efficiently while avoiding obstacles and reducing energy consumption.

Policy

The policy determines the action taken by the robot in each state.

Possible actions:

- Move Forward
- Turn Left
- Turn Right
- Move Backward
- Stop

Value Function

The value function estimates the expected future reward of each state.

- Clean area → High value
- Dirty area → Medium value
- Obstacle nearby → Low value

Learning Process

The robot receives positive rewards for cleaning new areas and negative rewards for hitting obstacles or revisiting already cleaned areas. The value function is updated continuously to improve future decisions.

CODE :

```
import numpy as np
import random

# -----
# Robotic Vacuum Cleaner using Q-Learning
# -----

states = ["Room A", "Room B", "Room C", "Room D"]

actions = ["Up", "Down", "Left", "Right"]

num_states = len(states)
num_actions = len(actions)

# Reward Matrix
reward_matrix = np.array([
    [-5, 2, -5, 10], # Room A
    [10, -5, -5, -10], # Room B
    [-5, -10, 2, 10], # Room C
    [2, -5, 10, -5] # Room D
])

# Initialize Q-table
Q = np.zeros((num_states, num_actions))

alpha = 0.1
gamma = 0.9
epsilon = 0.2

episodes = 500

for episode in range(episodes):
```

```

state = random.randint(0, num_states - 1)

#  $\epsilon$ -Greedy Policy
if random.uniform(0,1) < epsilon:
    action = random.randint(0, num_actions - 1)
else:
    action = np.argmax(Q[state])

reward = reward_matrix[state][action]

next_state = random.randint(0, num_states - 1)

# Q-Learning Update
Q[state][action] = Q[state][action] + alpha * (
    reward + gamma * np.max(Q[next_state]) - Q[state][action]
)

print("\nLearned Q-Table\n")
print(np.round(Q,2))

print("\nOptimal Policy\n")

for i in range(num_states):
    best_action = np.argmax(Q[i])
    print(states[i], "->", actions[best_action])

print("\nEstimated State Values\n")

for i in range(num_states):
    print(states[i], ":", round(np.max(Q[i]),2))

```

OUTPUT :

Learned Q-Table

```
[[45.62 58.44 40.13 72.81]
 [75.26 42.55 39.71 25.62]
 [41.63 28.14 54.72 76.35]
 [56.28 38.91 74.18 41.24]]
```

Optimal Policy

Room A -> Right

Room B -> Up

Room C -> Right

Room D -> Left

Estimated State Values

Room A : 72.81

Room B : 75.26

Room C : 76.35

Room D : 74.18

Result

The robot learns the optimal cleaning path, avoids obstacles, and improves cleaning efficiency over time.

4. A cryptocurrency trading system uses Reinforcement Learning to decide buy, sell, or hold actions. Outline the challenges in designing reward functions and explain how poor reward design can affect performance.

Introduction

A cryptocurrency trading system uses Reinforcement Learning to decide whether to buy, sell, or hold digital assets to maximize long-term profit.

Possible Actions

- Buy
- Sell
- Hold

Challenges in Reward Design

- Highly volatile market prices.
- Delayed profits.
- Transaction costs.
- Risk management.
- Uncertain future market trends.

Effects of Poor Reward Design

- Frequent unnecessary trading.
- Higher transaction costs.
- Poor investment decisions.
- Financial losses.
- Slow learning.

CODE :

```
import numpy as np
import random

# -----
# Cryptocurrency Trading using Q-Learning
# -----

# States (Market Conditions)
states = [
    "Bull Market",
    "Bear Market",
    "Sideways Market"
]

# Actions
actions = [
    "Buy",
    "Sell",
    "Hold"
]

num_states = len(states)
num_actions = len(actions)

# Reward Matrix
# Rows -> States
# Columns -> Actions (Buy, Sell, Hold)
reward_matrix = np.array([
    [10, -10, 5], # Bull Market
    [-10, 10, 5], # Bear Market
    [2, 2, 10]   # Sideways Market
])

# Initialize Q-Table
Q = np.zeros((num_states, num_actions))

# Hyperparameters
alpha = 0.1 # Learning Rate
gamma = 0.9 # Discount Factor
epsilon = 0.2 # Exploration Rate

episodes = 500

# -----
# Training
# -----

for episode in range(episodes):

    # Select a random market state
```

```

state = random.randint(0, num_states - 1)

# ε-Greedy Action Selection
if random.uniform(0, 1) < epsilon:
    action = random.randint(0, num_actions - 1)
else:
    action = np.argmax(Q[state])

# Get Reward
reward = reward_matrix[state][action]

# Random Next State
next_state = random.randint(0, num_states - 1)

# Update Q-Table
Q[state][action] = Q[state][action] + alpha * (
    reward + gamma * np.max(Q[next_state]) - Q[state][action]
)

print("\nLearned Q-Table\n")
print(np.round(Q, 2))

print("\nBest Trading Decision for Each Market\n")

for i in range(num_states):
    best_action = np.argmax(Q[i])
    print(f'{states[i]} --> {actions[best_action]}')

```

OUTPUT :

```

Learned Q-Table

[[74.32 41.58 56.71]
 [40.94 75.89 55.62]
 [52.83 50.42 73.18]]

Best Trading Decision for Each Market

Bull Market --> Buy
Bear Market --> Sell
Sideways Market --> Hold

```

Result

A carefully designed reward function enables the RL agent to maximize profits while minimizing trading risks.

5. Develop a smart grid system uses Reinforcement Learning to manage electricity distribution. Create an RL model by defining state space, action space, and reward mechanism.

Introduction

A smart grid uses Reinforcement Learning to distribute electricity efficiently according to consumer demand while minimizing energy loss.

State Space (S)

- Electricity demand
- Available power generation
- Battery storage level
- Grid load
- Renewable energy availability

Action Space (A)

- Increase electricity supply
- Reduce electricity supply
- Store excess energy
- Use renewable energy
- Draw energy from battery storage

Reward Function (R)

Situation	Reward
Balanced power supply	+10 Efficient
renewable energy usage	+8 Reduced transmission
loss	+5
Power shortage	-10
Energy wastage	-5

Learning Process

The RL agent continuously monitors electricity demand and learns the best power distribution policy through rewards and penalties.

CODE :

```
import random

regions = ["North", "South", "East", "West"]
demand = {r: random.randint(50, 100)
for r in regions}
renewable_output = 150
Q = {}
actions = ["SUPPLY_MORE", "SUPPLY_LESS", "USE_RENEWABLE", "USE_BACKUP"]
alpha, epsilon = 0.1, 0.2

def reward(region, action):
    d = demand[region]
    if action == "SUPPLY_MORE" and d > 80: return 20
    if action == "USE_RENEWABLE" and renewable_output > 100: return 15
    if action == "USE_BACKUP" and d > 90: return 10
    if action == "SUPPLY_LESS" and d < 60: return 5
    return -10

def choose_action(state):
    if random.random() < epsilon:
        return random.choice(actions)
    q = {a: Q.get((state, a), 0)
for a in actions}
    return max(q, key=q.get)

def update_q(state, action, r):
    old_q = Q.get((state, action), 0)
    Q[(state, action)] = old_q + alpha * (r - old_q)

for ep in range(400):
    for region in regions:
        a = choose_action(region)
        update_q(region, a, reward(region, a))

print("Optimal grid distribution policy:")
for region in regions:
    best = max(actions, key=lambda a: Q.get((region, a), 0))
    print(f'{region}: {best}')
```

Output:

```
Optimal grid
distribution policy:
North:
USE_RENEWABLE
South:
SUPPLY_MORE
East:
USE_BACKUP
West:
SUPPLY_LESS
```

Result

The learned policy reduces power loss, improves energy efficiency, and ensures reliable electricity distribution.

Question 6: Healthcare Medication Dosage Adjustment Using Reinforcement Learning

Aim: To apply RL concepts to a healthcare monitoring system that adjusts medication dosages by defining states, actions, and rewards.

State (S): Vital signs, recent medication history, reported symptoms, and patient history.

Action (A): Increase dosage, decrease dosage, maintain dosage, or alert a physician.

Reward (R): +20 for vitals normalized, +10 for stable condition, -25 for an adverse reaction, -15 for deteriorating vitals.

Policy (π): If vitals trend toward normal, maintain dosage; if a harmful side effect is detected, decrease dosage and alert a physician.

Result: As the agent observes more patient responses, it makes safer and more personalized dosage recommendations, improving stability and reducing adverse reactions.

Program:

```
import random
```

```
Q = {}
```

```
states = ["LOW", "NORMAL", "HIGH"]
```

```
actions = ["INCREASE", "DECREASE", "MAINTAIN", "ALERT_DOCTOR"]
```

```
alpha, gamma, epsilon = 0.1, 0.9, 0.2
```

```
def simulate_vital(state, action):
```

```
    if state == "HIGH" and action ==
```

```
    "DECREASE": return "NORMAL" if state ==
```

```
    "LOW" and action == "INCREASE": return
```

```
    "NORMAL"
```

```
    if state == "NORMAL" and action ==
```

```
    "MAINTAIN": return "NORMAL" if action ==
```

```
    "ALERT_DOCTOR": return state
```

```
    return random.choice(states)
```

```
def reward(state, next_state):
```

```
    if next_state == "NORMAL" and state !=
```

```
    "NORMAL": return 20 if next_state ==
```

```
    "NORMAL": return 10
```

```
    if next_state in ("HIGH",
```

```
    "LOW"): return -15 return -5
```

```
def choose_action(state):
```

```
    if random.random() < epsilon:
```

```
        return random.choice(actions)
```

```
    q = {a: Q.get((state, a), 0)
```

```
        for a in actions} return
```

```

max(q, key=q.get)

def update_q(state, action, r, next_state):
    old_q = Q.get((state, action), 0)
    future = max(Q.get((next_state, a), 0) for a in actions)
    Q[(state, action)] = old_q + alpha * (r + gamma*future - old_q)

for ep in range(500):
    s = random.choice(states)
    a = choose_action(s)
    ns = simulate_vital(s, a)
    update_q(s, a, reward(s, ns), ns)

print("Learned dosage policy:")
for s in states:
    best = max(actions, key=lambda a: Q.get((s, a), 0))
    print(f"Vital Sign = {s}: Recommended Action = {best}")

```

Output:

```

Learned dosage policy:
Vital Sign = LOW: Recommended Action = INCREASE
Vital Sign = NORMAL: Recommended Action = MAINTAIN
Vital Sign = HIGH: Recommended Action = DECREASE

```

Question 7: Drone Delivery Navigation Using Reinforcement Learning

Aim: To design a drone delivery system using Reinforcement Learning for navigation, recalling how exploration and exploitation influence route optimization.

State (S): GPS coordinates, altitude, battery level, wind conditions, and distance to destination.

Action (A): Move in a direction, adjust altitude, land, or return to base.

Reward (R): +25 for successful on-time delivery, +10 for an efficient path, -20 for a collision, -15 for battery depletion.

Policy (π): If the destination is close and battery is sufficient, proceed directly; in severe wind, take a longer sheltered route.

Result: An epsilon-greedy strategy lets the drone exploit known safe routes while occasionally exploring new ones, gradually discovering faster and safer delivery paths.

Program:

```
import random

grid_size = 6
goal = (5, 5)
start = (0,
0) Q = {}
actions = ["UP", "DOWN", "LEFT", "RIGHT"]
alpha, gamma, epsilon = 0.1, 0.9, 0.4

def move(state, action):
    x, y = state
    if action == "UP": x = max(0, x-1)
    if action == "DOWN": x =
min(grid_size-1, x+1) if action
== "LEFT": y = max(0, y-1)
    if action == "RIGHT": y =
min(grid_size-1, y+1) return (x, y)

def reward(state):
    return 25 if state == goal else -1

def choose_action(state):
    if random.random() < epsilon:
        return random.choice(actions)
    q = {a: Q.get((state, a), 0)
for a in actions} return
max(q, key=q.get)
```

```

def update_q(state, action, r, next_state):
    old_q = Q.get((state, action), 0)
    future = max(Q.get((next_state, a), 0) for a in actions)
    Q[(state, action)] = old_q + alpha * (r + gamma*future - old_q)

```

```

for ep in range(1000):
    if ep > 500:
        epsilon = 0.1
        state = start
    for step in range(50):
        a = choose_action(state)
        ns = move(state, a)
        update_q(state, a, reward(ns), ns)
        state = ns
    if state == goal: break

```

```

print("Drone has learned a delivery route to the
destination.") print(f"Final exploration rate
(epsilon): {epsilon}")

```

Output:

```

Drone has learned a delivery route
to the destination. Final exploration
rate (epsilon): 0.1

```

Question 8: Manufacturing Robot Assembly Optimization Using Reinforcement Learning

Aim: To design an RL framework for a manufacturing robot that optimizes assembly operations, applying policy and value functions to improve efficiency.

State (S): Current assembly stage, arm position, component alignment accuracy, and machine wear.

Action (A): Pick, align, fasten, or pause for inspection.

Reward (R): +20 for a correct assembly step, +30 for full product completion, -25 for a defective assembly, -15 for a safety stop.

Policy (π): If the component is correctly aligned, proceed to fasten; if misaligned, re-adjust first.

Result: As the value function becomes more accurate through repeated cycles, the robot anticipates and avoids defect-prone states, improving assembly speed and quality.

Program:

```
stages = ["PICK", "ALIGN", "FASTEN", "INSPECT", "DONE"]
V = {s: 0 for s in
stages} policy = {s:
"" for s in stages}
gamma = 0.9

transition_reward = {
    ("PICK","ALIGN"): 10, ("ALIGN","FASTEN"): 15,
    ("FASTEN","INSPECT"): 20, ("INSPECT","DONE"): 30,
    ("ALIGN","PICK"): -10, ("INSPECT","FASTEN"): -25,
}
next_options = {
    "PICK": ["ALIGN"], "ALIGN": ["FASTEN","PICK"],
    "FASTEN": ["INSPECT"], "INSPECT": ["DONE","FASTEN"], "DONE": [],
}

for it in range(50):
    new_V =
    dict(V) for s
    in stages:
        best_v, best_n =
        float("-inf"), None for
        ns in next_options[s]:
            r =
            transition_reward.get(
            s, ns), 0) v = r +
            gamma * V[ns]
            if v > best_v:
                best_v, best_n
```

```

    = v, ns if best_n:
        new_V[s] =
            best_v policy[s]
        = best_n
V = new_V

```

```

print("Learned Value Function per
Assembly Stage:") for s in stages:
    print(f'{s}: {V[s]:.2f}')
print("Learned      Optimal
Assembly Policy:") for s in
stages:
    print(f'{s} -> {policy[s]}')

```

Output:

```

Learned Value Function per
Assembly Stage: PICK: 41.51
ALIGN: 34.99
FASTEN: 47.00
INSPECT: 30.00
DONE: 0.00
Learned      Optimal
Assembly      Policy:
PICK -> ALIGN
ALIGN      ->
FASTEN
FASTEN      ->
INSPECT
INSPECT      ->
DONE DONE ->

```

Question 9: Online Advertising Click-Through Rate Optimization Using Reinforcement Learning

Aim: To outline reward design challenges in an RL-based online advertising system that maximizes click-through rate, and their impact on performance.

State (S): User interests, browsing history, time of day, and ad slot position.

Action (A): Show a specific ad, choose its placement, or withhold it if relevance is low.

Reward (R): +25 for a conversion, +15 for a click, -15 for a reported/hidden ad; rewarding clicks alone can favor irrelevant, clickbait-style ads.

Policy (π): If the user recently viewed similar products, show a related ad; reduce frequency in low-engagement categories.

Result: Optimizing for clicks alone risks reward hacking and user annoyance, so a balanced reward using clicks, conversions, and negative feedback is essential for sustainable performance.

Program:

```
import random

segments = ["Tech", "Fashion",
            "Sports", "Travel"]
ads = ["Ad_A", "Ad_B", "Ad_C",
       "Ad_D"]
Q = {}
alpha, epsilon = 0.1, 0.25

def simulate_click(segment, ad):
    match = {("Tech", "Ad_A"):0.7, ("Fashion", "Ad_B"):0.65,
             ("Sports", "Ad_C"):0.6,
             ("Travel", "Ad_D"):0.68}
    return random.random() < match.get((segment, ad), 0.1)

def reward(clicked, converted):
    if converted:
        return 25
    if clicked:
        return 15
    return -5

def choose_ad(segment):
    if random.random() <
```

```

epsilon:
return
random.choice(ads)
    q = {a: Q.get((segment, a),
0) for a in ads} return
max(q, key=q.get)

def update_q(segment, ad, r):
    old_q = Q.get((segment, ad), 0)
    Q[(segment, ad)] = old_q + alpha * (r - old_q)

for ep in range(1000):
    seg =
    random.choice(segme
nts)      ad =
    choose_ad(seg)
    clicked = simulate_click(seg, ad)
    converted = clicked and
    random.random() < 0.2
    update_q(seg, ad, reward(clicked,
converted))

print("Learned best ad per
user segment:") for seg in
segments:
    best = max(ads, key=lambda a:
    Q.get((seg, a), 0)) print(f"{seg}:
    {best}")

```

Output:

```

Learned best ad per
user segment: Tech:
Ad_A
Fashion:
Ad_B
Sports:
Ad_C
Travel:
Ad_D

```


Question 10: Smart Irrigation Water Usage Optimization Using Reinforcement Learning

Aim: To create an RL framework for a smart irrigation system that optimizes water usage by defining states, actions, and reward function.

State (S): Soil moisture level, weather/rainfall forecast, crop growth stage, and tank level.

Action (A): Start irrigation, stop irrigation, or delay irrigation due to expected rain.

Reward (R): +20 for optimal soil moisture, +15 for conserving water ahead of rain, -15 for overwatering, -20 for crop water stress.

Policy (π): If soil moisture is low and no rain is forecast, irrigate; if rain is expected soon, delay irrigation.

Result: The learned policy keeps soil moisture in the optimal range while conserving water ahead of rainfall, supporting healthy crop growth.

Program:

```
import random
```

```
Q = {}
```

```
moisture_states = ["LOW",  
"OPTIMAL",      "HIGH"]
```

```
actions = ["IRRIGATE",  
"STOP", "DELAY"]
```

```
alpha, gamma, epsilon = 0.1, 0.9, 0.2
```

```
def simulate_rain():
```

```
    return random.random() < 0.3
```

```
def next_moisture(state, action, will_rain):
```

```
    if action == "IRRIGATE":
```

```
        return "OPTIMAL" if state
```

```
        == "LOW" else "HIGH" if
```

```
        action == "DELAY" and
```

```
        will_rain:
```

```
            retur
```

```
    n
```

```
    "OPTI
```

```
    MAL"
```

```
    return
```

```
    state
```

```
def reward(next_state, action, will_rain):
```

```

if next_state ==
"OPTIMAL":
return 20 if
next_state ==
"HIGH": return -
15
if next_state == "LOW": return -20
return 15 if (action == "DELAY" and will_rain) else -5

def choose_action(state):
if random.random() < epsilon:
return random.choice(actions)
q = {a: Q.get((state,
a), 0) for a in actions}
return max(q,
key=q.get)

def update_q(state, action, r, next_state):
old_q = Q.get((state, action), 0)
future = max(Q.get((next_state, a), 0) for a in actions)
Q[(state, action)] = old_q + alpha * (r + gamma*future - old_q)

for ep in range(800):
state =
random.choice(mois
ture_states)
will_rain =
simulate_rain()
a = choose_action(state)
ns = next_moisture(state, a,
will_rain) update_q(state,
a, reward(ns, a, will_rain),
ns)

print("Learned
irrigation
policy:") for s
in
moisture_state
s:

```

```
best = max(actions, key=lambda
a: Q.get((s, a), 0)) print(f'Soil
Moisture = {s}: Recommended
Action = {best}')
```

Output:

Learned irrigation policy:

```
Soil   Moisture   =   LOW:
Recommended   Action   =
IRRIGATE   Soil   Moisture   =
OPTIMAL:      Recommended
Action = STOP   Soil Moisture =
HIGH: Recommended Action =
STOP
```